Programación Móvil

Tema 4

Ciclo de Vida y Componentes Fundamentales

TEMAS

1. El concepto de Activity y su ciclo de vida.
2. Introducción a la arquitectura de componentes.
3. Activity en Android vs. UIViewController en iOS.
4. Comparación de métodos de ciclo de vida.

OBJETIVOS

1. Explicar el ciclo de vida de una Activity y el propósito de onCreate, onStart, onResume, onPause, onStop y onDestroy.
2. Aplicar técnicas de persistencia de estado para manejar cambios de configuración como la rotación.
3. Describir la arquitectura de componentes (UI / Domain / Data) y justificar la separación de responsabilidades.
4. Comparar Activity (Android) y UIViewController (iOS).

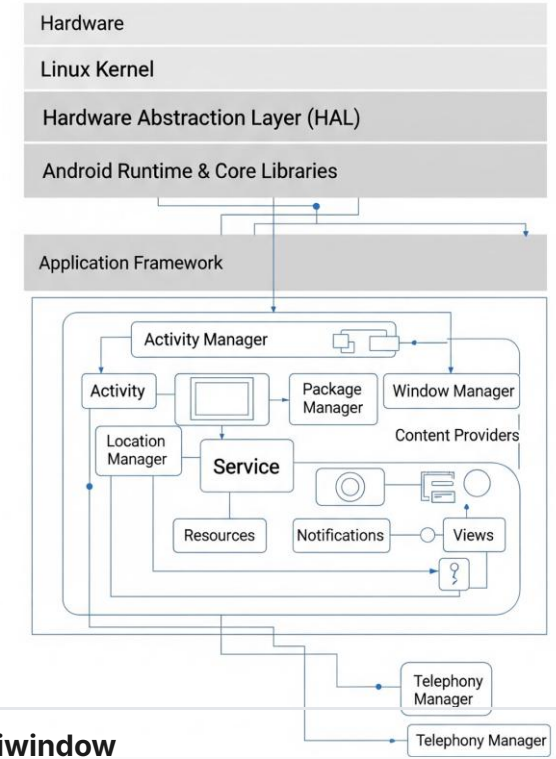
¿Por qué importa el ciclo de vida?

Las apps modernas son **ecosistemas de componentes efímeros**.

El SO puede destruir y recrear componentes por falta de **memoria** o cambios de **configuración**.

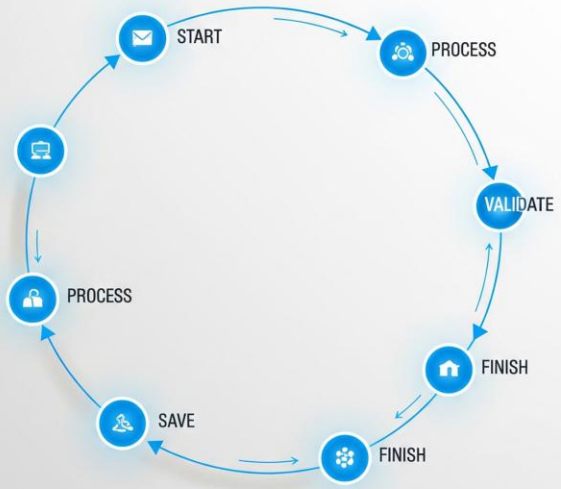
Esto tiene un **impacto directo** en la UX, el rendimiento y la persistencia de datos.

"La evolución de los sistemas operativos móviles ha transformado la noción de aplicación, pasando de ser un ejecutable lineal a un ecosistema complejo de componentes efímeros y reactivos."



Ejemplos reales: rotación, multiwindow

Ciclo de vida de Activity — vista general



Métodos (Callbacks):

onCreate & onStart
Inicialización, cargar el layout y UI visible.

onResume
Estado de interactividad plena con el usuario.

onPause & onStop
Degradación y liberación activa de recursos.

onDestroy & onRestart
Limpieza final o reentrada al ciclo visible.

Callbacks clave y buenas prácticas

onCreate

Inicializar ViewModel y setContentView( ) o UI de Compose (setContent{...}).

onResume

Reanudar sensores, animaciones y listeners de interacción.

onPause

Operaciones ligeras. No bloquear la UI para transiciones rápidas.

onStop

Liberar recursos pesados (GPS, conexiones, hilos).

Example.kt – Activity Callbacks

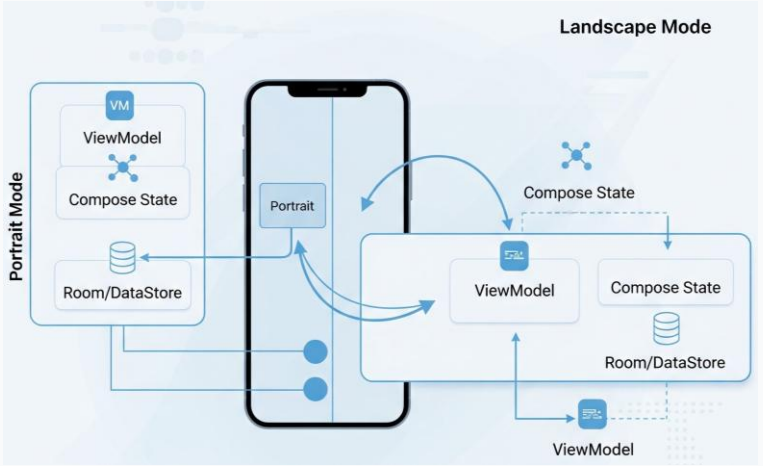
```
override fun onStart() {
super.onStart()
// Register heavy listeners
sensorManager.registerListener(...)
}

override fun onStop() {
// Release resources
sensorManager.unregisterListener(...)
super.onStop()
}

override fun onPause() {
// Quick state save only
super.onPause()
saveTripState()
}
```

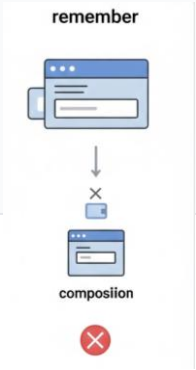
Estrategias para conservar datos en cambios de configuración

Herramienta	Uso y Alcance
ViewModel + SavedStateHandle	ViewModel Sobrevive a rotación. SavedStateHandle necesario para Process Death.
Compose State (Saveable / Retain)	rememberSaveable para rotación/muerte; retain (API 36) para objetos pesados para rotación.
Room / DataStore	Persistencia física definitiva para datos críticos y volúmenes grandes de información.



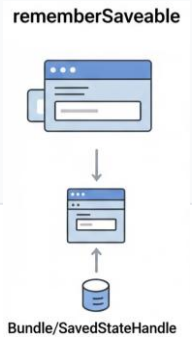
Nota: La UI debe derivarse siempre de modelos de datos persistentes.

Estado en Compose — remember vs rememberSaveable vs retain



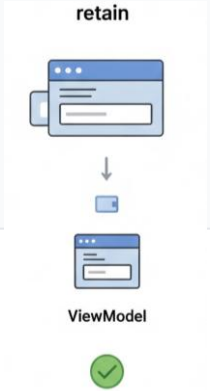
Persiste el estado durante la **recomposición**. Se pierde en cambios de configuración (rotación).

Uso: Cálculos internos efímeros.



Sobrevive a **rotación** y muerte del proceso. Requiere datos **serializables** (**Bundle**).

Uso: Inputs de texto, scroll position, selección de pestañas.



Mantiene **objetos pesados** en memoria durante la rotación. No requiere serialización.

Uso: ExoPlayer, Bitmaps, Grafos, objetos complejos.

"No todo el estado debe sobrevivir igual. Cada herramienta tiene un propósito."

Recomendación: Usar **rememberSaveable** para inputs de UI; **retain** para objetos no serializables pesados.

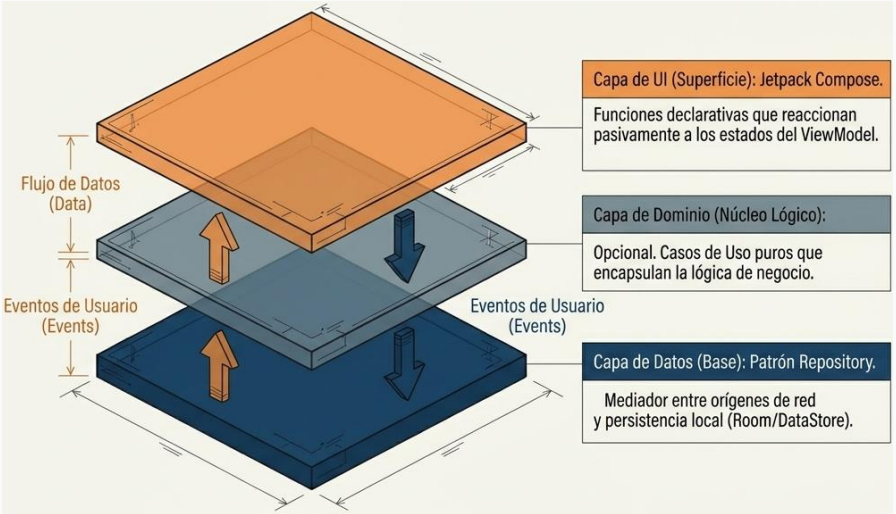
Nota: retain es novedad en Android 16 (API 36) para evitar fugas y recargas costosas.

Estructura multicapa recomendada por Google

UI LAYER
Compose / Activities / Fragments
Transforma datos en representaciones visuales interactivas.

DOMAIN LAYER
Casos de Uso (Use Cases)
Lógica de negocio pura y encapsulamiento de tareas específicas.

DATA LAYER
Repositorios, Room, APIs
Cimiento de la app: obtención, persistencia y sincronización.



Separar UI, lógica de negocio y acceso a datos para lograr apps más mantenibles, escalables y robustas.

Los cuatro pilares del sistema Android



ACTIVITY

Interfaz de Usuario

Pantalla con la que el usuario interactúa.

Punto de entrada para la interacción humana. Gestiona el foco y la visibilidad.



SERVICE

Operaciones de Fondo

Ejecuta tareas en segundo plano.

API 36: Restricciones severas para inicios desde segundo plano.



BROADCAST RECEIVER

Eventos del Sistema

Escucha eventos globales del SO.

API 36: Prioridades limitadas en anuncios globales para estabilidad.



CONTENT PROVIDER

Gestión de Datos

Facilita la compartición segura de datos entre aplicaciones.

Comparación Android vs iOS — responsabilidades y lifecycle mapping

ETAPA	ANDROID (ACTIVITY)	IOS (VIEWCONTROLLER)
Creación	`onCreate()`	`viewDidLoad()`
Visibilidad	`onStart()`	`viewWillAppear()`
Interactividad	`onResume()`	`viewDidAppear()`
Ocultación	`onPause()` / `onStop()`	`viewWillDisappear()` / `viewDidDisappear()`
Destrucción	`onDestroy()`	`deinit`

Innovaciones en Android 16 que afectan el ciclo de vida

PREDICTIVE BACK

Fin de onBackPressed

Transición obligatoria a OnBackInvokedDispatcher para navegación fluida basada en gestos.

EFICIENCIA ENERGÉTICA

Cuotas de ejecución dinámicas

Prioridad de Jobs ajustada según el estado Top Resumed; mayor cuota mientras es interactiva.

GESTIÓN DE ESTADO

retain API (API 36)

Supervivencia de objetos no serializables y pesados ante rotación sin recargas costosas.

Práctica — Observar el ciclo de vida

INSTRUCCIONES PASO A PASO

01 **Configuración Base**
Crear una Activity simple con un contador y logs en cada callback del ciclo de vida.

02 **Persistencia en Memoria**
Implementar ViewModel para mantener el estado del contador ante cambios de configuración.

03 **Prueba de Rotación**
Rotar el dispositivo; observar los logs de destrucción/recreación y el valor del contador.

04 **Muerte de Proceso (Process Death)**
Activar "Don't keep activities", forzar salida y reentrada para probar SavedStateHandle.

CRITERIOS DE ÉXITO

- El contador persiste visualmente tras la rotación de pantalla.
- SavedStateHandle restaura el valor tras simular la muerte del proceso.

Práctica — Formulario con persistencia robusta

REQUERIMIENTOS TÉCNICOS

Construir una pantalla de formulario con varios campos (texto, selección, scroll) que:

- **Persistencia de UI:** Mantenimiento de estado tras rotación (ViewModel + SavedStateHandle o rememberSaveable).
- **Almacenamiento Local:** Uso de Room o DataStore para guardar borradores de forma automática.
- **Navegación Moderna:** Implementación de manejo de back predictivo (Predictive Back) compatible con API 36.

ENTREGABLE

Los solicitados en el EVA

CRITERIOS DE EVALUACIÓN

- Persistencia correcta de todos los campos.
- Manejo eficiente del lifecycle.
- Código limpio y arquitectura sugerida.



Preguntas de reflexión (Contestar en EVA)

01 ¿Por qué Android recrea Activities en cambios de configuración?

02 ¿Cuándo es preferible usar ViewModel vs persistencia local?

03 ¿Qué riesgos trae mover lógica pesada a callbacks de UI?

04 ¿Cómo cambia la navegación con Predictive Back?

05 ¿Qué diferencias prácticas hay entre onDestroy y deinit?



Conclusiones y lecturas recomendadas

RESUMEN FINAL

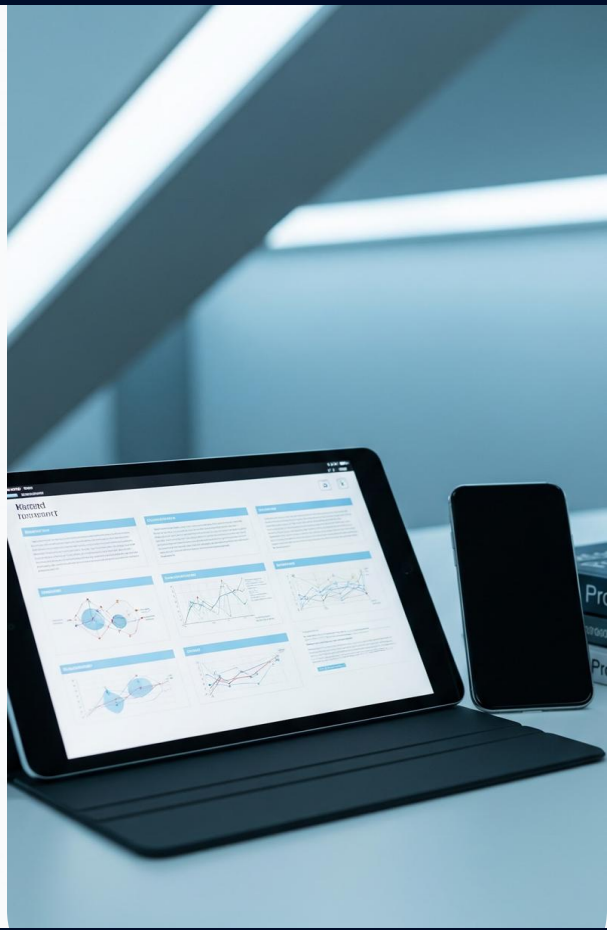
- **Activity como componente efímero:** Siempre diseñe la interfaz de usuario basándose en datos persistentes y flujos reactivos.
- **Estrategia de Estado:** Combine ViewModel con SavedStateHandle y APIs de estado de Compose según la complejidad del caso.
- **Adopción Android 16:** Implemente las nuevas APIs de navegación y persistencia pesada para asegurar compatibilidad futura.

DOCUMENTACIÓN

Guía oficial de Arquitectura de Apps Android y Ciclo de Vida de Activities.

RECURSOS WEB

Codelabs de Android 16: Implementando Predictive Back y APIs de persistencia.



Snippets útiles (Kotlin / Compose / SavedStateHandle)

```kotlin
// Lifecycle Observer
class MyObserver : DefaultLifecycleObserver {
override fun onResume(owner: LifecycleOwner) {
Log.d("LOG", "Resumed")
}
}
```

```kotlin
// rememberSaveable

var name by rememberSaveable {
mutableStateOf("")
}
TextField(value = name, onValueChange = { name = it })
```

```kotlin
// SavedStateHandle en ViewModel

class MyViewModel(state: SavedStateHandle) : ViewModel() {
val userId = state.getStateFlow("user_id", "default")
fun updateId(newId: String) {
state["user_id"] = newId
}
}
```